Агрегатные функции

Задание:

- 1.1. Рассчитайте общий доход от всех операций. - 1.2. Найдите средний доход с одной сделки. - 1.3. Определите общее количество проданной продукции. - 1.4. Подсчитайте количество уникальных пользователей, совершивших покупку.

Листинг скрипта

```
USE homework6;

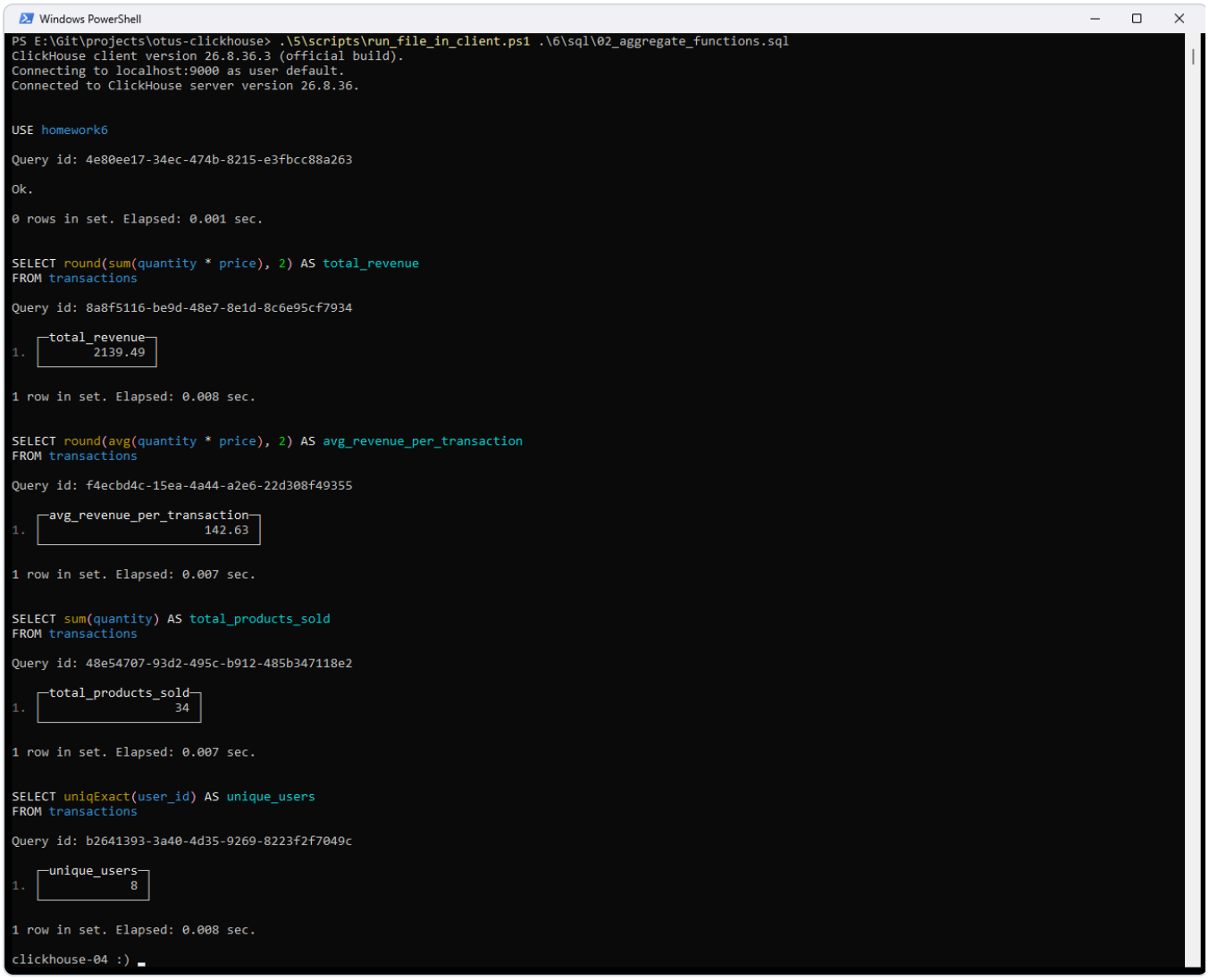
-- Общий доход от всех операций
SELECT round(sum(quantity * price), 2) AS total_revenue
FROM transactions;

-- Средний доход с одной сделки
SELECT round(avg(quantity * price), 2) AS avg_revenue_per_transaction
FROM transactions;

-- Общее количество проданной продукции
SELECT sum(quantity) AS total_products_sold
FROM transactions;

-- Количество уникальных пользователей, совершивших покупку
SELECT uniqExact(user_id) AS unique_users
FROM transactions;
```

Скриншот результата



Функции для работы с типами данных

Задание:

- 2.1. Преобразуйте `transaction_date` в строку формата `YYYY-MM-DD`. - 2.2. Извлеките год и месяц из `transaction_date`. - 2.3. Округлите `price` до ближайшего целого числа. - 2.4. Преобразуйте `transaction_id` в строку.

Листинг скрипта

```
USE homework6;

-- Преобразование transaction_date в строку формата YYYY-MM-DD
SELECT
    transaction_id,
```

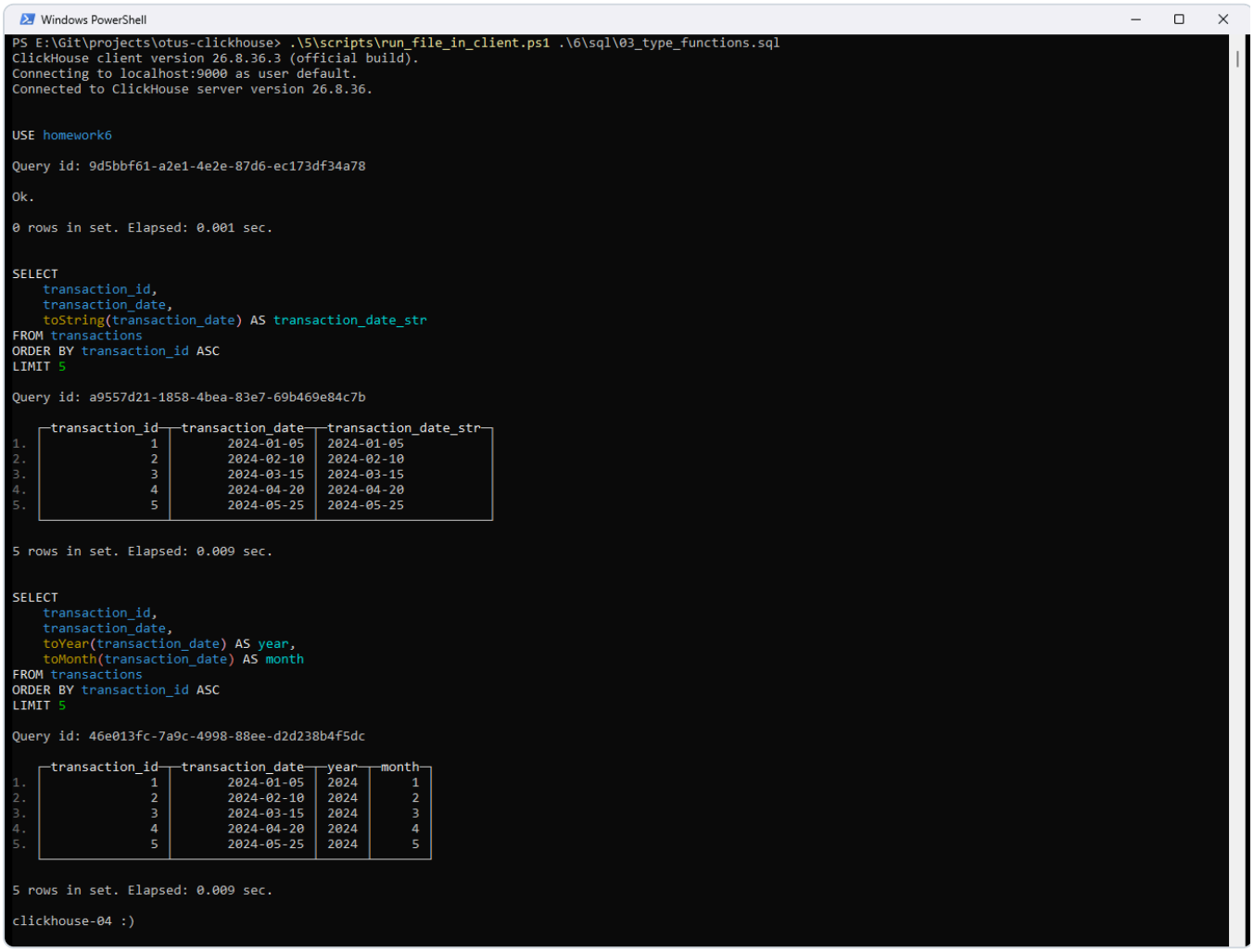
```
    transaction_date,
    toString(transaction_date) AS transaction_date_str
FROM transactions
ORDER BY transaction_id
LIMIT 5;

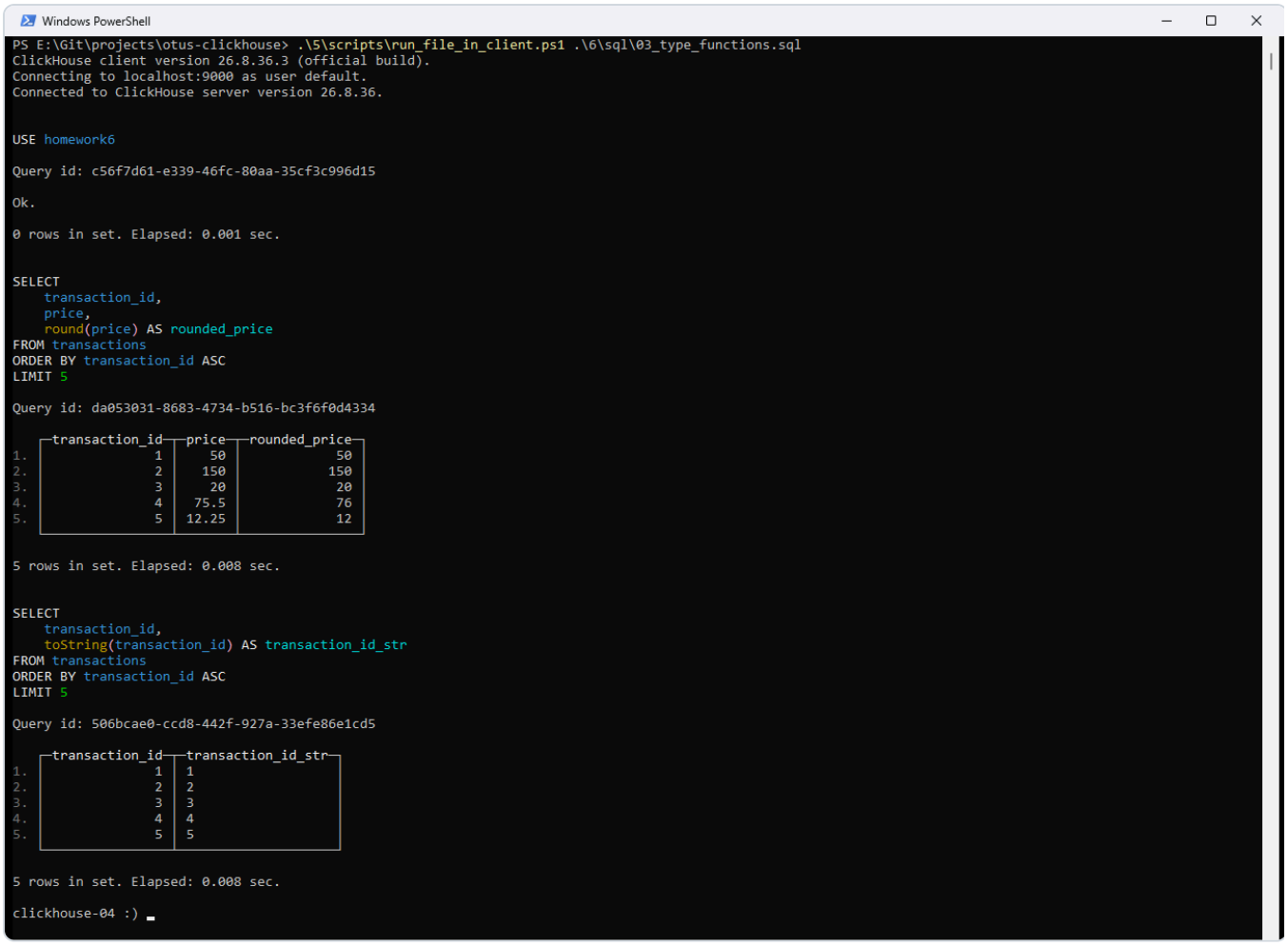
-- Извлечение года и месяца из transaction_date
SELECT
    transaction_id,
    transaction_date,
    toYear(transaction_date) AS year,
    toMonth(transaction_date) AS month
FROM transactions
ORDER BY transaction_id
LIMIT 5;

-- Округление price до ближайшего целого числа
SELECT
    transaction_id,
    price,
    round(price) AS rounded_price
FROM transactions
ORDER BY transaction_id
LIMIT 5;

-- Преобразование transaction_id в строку
SELECT
    transaction_id,
    toString(transaction_id) AS transaction_id_str
FROM transactions
ORDER BY transaction_id
LIMIT 5;
```

Скриншот результата





User-Defined Functions (UDFs)

Задание:

- 3.1. Создайте простую UDF для расчёта общей стоимости транзакции. - 3.2. Используйте созданную UDF для расчёта общей цены для каждой транзакции. - 3.3. Создайте UDF для классификации транзакций на «высокоценные» и «малоценные» на основе порогового значения (например, 100). - 3.4. Примените UDF для категоризации каждой транзакции.

Листинг скрипта

```
-- User-Defined Functions (UDF)
USE homework6;

-- Простая UDF для расчёта общей стоимости транзакции
CREATE OR REPLACE FUNCTION total_price AS (quantity, price) ->
    round(quantity * price, 2);

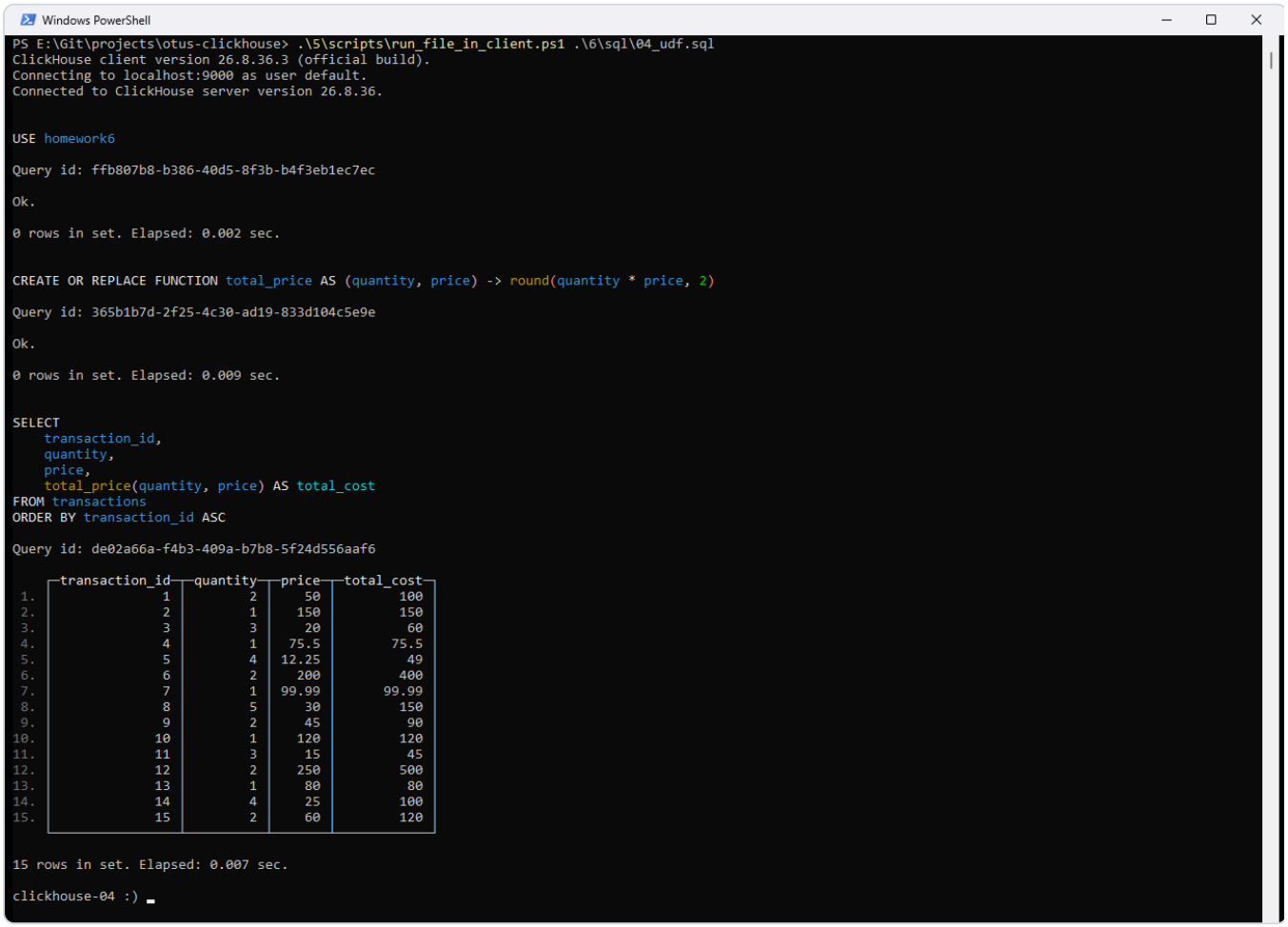
-- Использование UDF для расчёта общей цены каждой транзакции
SELECT
```

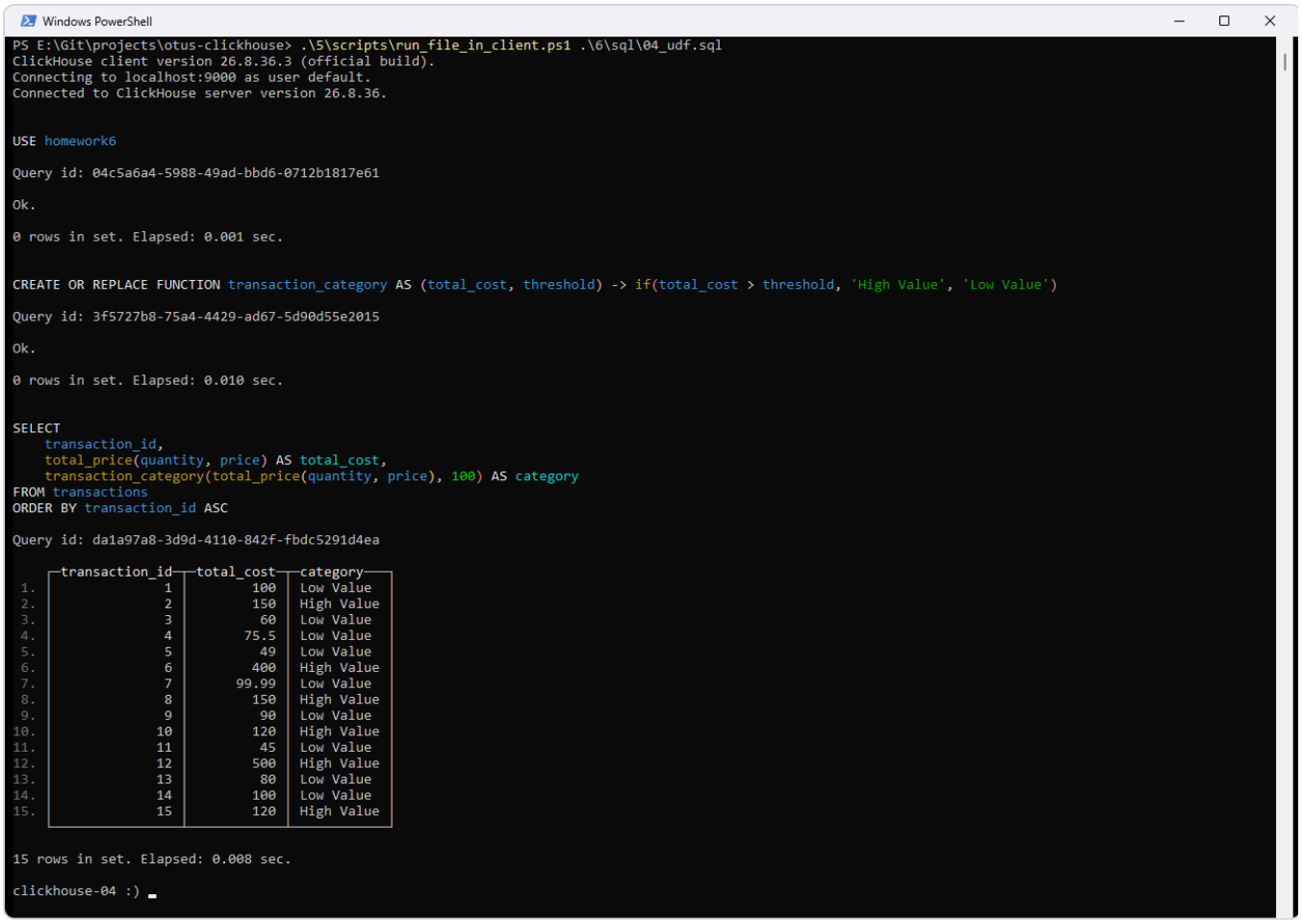
```
    transaction_id,
    quantity,
    price,
    total_price(quantity, price) AS total_cost
FROM transactions
ORDER BY transaction_id;

-- UDF для классификации транзакций по пороговому значению
CREATE OR REPLACE FUNCTION transaction_category AS (total_cost, threshold) ->
    if(total_cost > threshold, 'High Value', 'Low Value');

-- Категоризация каждой транзакции
SELECT
    transaction_id,
    total_price(quantity, price) AS total_cost,
    transaction_category(total_price(quantity, price), 100) AS category
FROM transactions
ORDER BY transaction_id;
```

Скриншот результата




Windows PowerShell
PS E:\Git\projects\otus-clickhouse> .\5\scripts\run_file_in_client.ps1 .\6\sql\04_udf.sql
ClickHouse client version 26.8.36.3 (official build).
Connecting to localhost:9000 as user default.
Connected to ClickHouse server version 26.8.36.
USE homework6
Query id: 04c5a6a4-5988-49ad-bbd6-0712b1817e61
Ok.
0 rows in set. Elapsed: 0.001 sec.
CREATE OR REPLACE FUNCTION transaction_category AS (total_cost, threshold) -> if(total_cost > threshold, 'High Value', 'Low Value')
Query id: 3f5727b8-75a4-4429-ad67-5d90d55e2015
Ok.
0 rows in set. Elapsed: 0.010 sec.
SELECT
transaction_id,
total_price(quantity, price) AS total_cost,
transaction_category(total_price(quantity, price), 100) AS category
FROM transactions
ORDER BY transaction_id ASC
Query id: da1a97a8-3d9d-4110-842f-fbdc5291d4ea
transaction_id total_cost category
1. 1 100 Low Value
2. 2 150 High Value
3. 3 60 Low Value
4. 4 75.5 Low Value
5. 5 49 Low Value
6. 6 400 High Value
7. 7 99.99 Low Value
8. 8 150 High Value
9. 9 90 Low Value
10. 10 120 High Value
11. 11 45 Low Value
12. 12 500 High Value
13. 13 80 Low Value
14. 14 100 Low Value
15. 15 120 High Value
15 rows in set. Elapsed: 0.008 sec.
clickhouse-04 :)